PDF TRANSCRIPT

TypeScript по этапам

Собранный диалог-методичка: от старта по этапам до финальной части про Vue 3 + Composition API + TypeScript.

Содержимое оформлено как читабельный transcript с сохранением структуры этапов, пояснений и примеров кода.

Диапазон: от сообщения пользователя «пойдем по этапам, сначала Этап 1. Основа синтаксиса типов» до последней фразы ответа про сборку цельной методички по всем 9 этапам.

Оформление: сохранены роли собеседников, заголовки этапов, списки и кодовые примеры.

TypeScript по этапам — диалог

Формат: transcript-версия нашего диалога, собранная от сообщения **«пойдем по этапам, сначала Этап 1. Основа синтаксиса типов»** до финальной фразы ответа про **Vue 3 + Composition API + TS**.

Пользователь

пойдем по этапам, сначала Этап 1. Основа синтаксиса типов

ChatGPT

Отлично. Идём по-нормальному, как на обучении: сначала теория, потом маленькие примеры, потом практика.

Этап 1. Основа синтаксиса типов

На этом этапе цель одна:
научиться читать и писать простые типы в TypeScript.

Пока не лезем в сложные штуки вроде generics, utility types и сложной типизации Vue.
Сейчас тебе нужно понять базовый язык типов.

---

1. Что такое тип в TypeScript

Тип отвечает на вопрос:

что за значение лежит в переменной?

Например:

```
const userName: string = 'Dmitriy'
const age: number = 25
const isAdmin: boolean = true
```

Здесь:

- `userName` — строка
- `age` — число

- `isAdmin` — логическое значение

Вот эта часть:

```
: string
```

называется **аннотацией типа**.

Общая форма:

```
const variableName: Тип = значение
```

---

2. Базовые примитивные типы

`string`

```
const city: string = 'Moscow'
```

Строка.

`number`

```
const count: number = 10
const price: number = 99.5
```

В TS нет отдельного типа `int` или `float`. Всё это `number`.

`boolean`

```
const isOpen: boolean = false
```

Только `true` или `false`.

`null`

```
const data: null = null
```

Осознанно пустое значение.

`undefined`

```
const value: undefined = undefined
```

Значение не определено.

---

3. Type inference — когда TS сам понимает тип

Очень важная тема.

Ты не обязан везде писать тип руками.

```
const name = 'Dima'
const age = 30
const active = true
```

TypeScript сам выведет:

- `name` → `string`
- `age` → `number`
- `active` → `boolean`

Это называется **вывод типа** или **type inference**.

То есть вот так чаще всего писать не надо:

```
const name: string = 'Dima'
```

Потому что TS и так это понимает.

Нормальный стиль:

```
const name = 'Dima'
```

Тип руками пишут там, где это реально нужно:

- в параметрах функций;
- в объектах со сложной структурой;
- когда TS выводит тип недостаточно точно;
- когда ты описываешь контракт данных.

---

4. `let`, `const` и поведение типов

Смотри:

```
const role = 'admin'
```

Для `const` TypeScript часто хранит более точное значение.

А здесь:

```
let role = 'admin'
```

значение можно поменять, поэтому TS мыслит шире: это просто `string`.

Пример:

```
const status = 'ok'
let message = 'hello'
```

- `status` ближе к конкретному значению `'ok'`
- `message` — просто `string`

Пока для базы тебе достаточно понимать так:

- `const` — значение не меняется
- `let` — значение можно переопределить
- TypeScript это учитывает

---

5. Массивы

Простой массив

```
const numbers: number[] = [1, 2, 3]
const names: string[] = ['Ann', 'Bob', 'Max']
```

Читается так:

- `number[]` — массив чисел
- `string[]` — массив строк

Альтернативная запись

```
const numbers: Array<number> = [1, 2, 3]
```

Это то же самое.

Но в обычном коде чаще пишут:

```
number[]
```

Смешанный массив

```
const values: (string | number)[] = ['a', 1, 'b', 2]
```

Это массив, где каждый элемент может быть либо строкой, либо числом.

---

6. Объекты

Объект можно типизировать прямо в месте объявления.

```
const user: { id: number; name: string } = {
  id: 1,
  name: 'Dmitriy',
}
```

Здесь объект обязан иметь:

- `id` типа `number`
- `name` типа `string`

Если написать так:

```
const user: { id: number; name: string } = {
  id: 1,
}
```

будет ошибка, потому что нет `name`.

Необязательное поле

```
const user: { id: number; name: string; phone?: string } = {
  id: 1,
  name: 'Dmitriy',
}
```

Знак `?` означает: поле может быть, а может отсутствовать.

То есть допустимы оба варианта:

```
const user1: { id: number; name: string; phone?: string } = {
  id: 1,
  name: 'Dima',
}
```

```
const user2: { id: number; name: string; phone?: string } = {
  id: 1,
  name: 'Dima',
  phone: '+79999999999',
}
```

---

7. Специальный момент: тип поля и отсутствие поля — не одно и то же

Вот это:

```
phone?: string
```

означает: свойства может не быть вообще.

А вот это:

```
phone: string | undefined
```

означает: свойство обязательно существует, но внутри него может быть `undefined`.

Разница важная.

---

8. Union types — значение может быть нескольких типов

Это одна из самых полезных вещей в TS.

```
let id: string | number

id = 'abc'
id = 123
```

Читается как:
`id` может быть либо `string`, либо `number`.

Ещё пример:

```
const result: 'success' | 'error' = 'success'
```

Здесь значение ограничено только двумя строками.

---

9. Literal types — конкретные значения как типы

Можно ограничить переменную не просто типом `string`, а конкретными строками.

```
let theme: 'light' | 'dark' = 'light'
```

Так можно:

```
theme = 'dark'
```

Так нельзя:

```
theme = 'blue'
```

Это очень полезно для:

- статусов;
- ролей;
- режимов;
- вкладок;
- состояний интерфейса.

---

10. `any` — опасный тип

```
let value: any = 'hello'
value = 123
value = true
value.test().abc.def
```

`any` отключает проверку типов.

То есть TypeScript как будто говорит:
“Ладно, я больше ничего не проверяю”.

Для новичка это ловушка.
Кажется удобно, но на практике это ломает смысл TS.

Правило: **используй `any` только как временную заглушку, если вообще без него никак.**

---

11. `unknown` — безопаснее, чем `any`

```
let value: unknown = 'hello'
```

`unknown` означает:
“Я пока не знаю, что это за тип”.

Но в отличие от `any`, ты не можешь сразу делать что угодно.

Например, так нельзя:

```
let value: unknown = 'hello'
value.toUpperCase()
```

Сначала надо проверить тип:

```
let value: unknown = 'hello'

if (typeof value === 'string') {
  console.log(value.toUpperCase())
}
```

Для будущей работы это очень важная привычка.

---

12. `null` и `undefined` в реальном коде

Очень часто будут встречаться такие случаи:

```
const selectedUser: string | null = null
```

Это значит:

- либо там строка,
- либо `null`

Пример:

```
let currentTab: string | null = null

currentTab = 'settings'
currentTab = null
```

Это суперчастая история для фронта.

---

13. Разница между “тип переменной” и “значение переменной”

Смотри:

```
const age: number = 25
```

- `number` — тип
- `25` — значение

Смотри ещё:

```
const theme: 'dark' | 'light' = 'dark'
```

- `'dark' | 'light'` — тип
- `'dark'` — значение

Это надо чётко разделять в голове.

---

14. Самые частые ошибки новичка

Ошибка 1. Путают `string` и `String`

Правильно:

```
const name: string = 'Dima'
```

Неправильно:

```
const name: String = 'Dima'
```

Нужно использовать маленькие примитивные типы:

- `string`
- `number`
- `boolean`

а не:

- `String`
- `Number`
- `Boolean`

Ошибка 2. Ставят `any` везде

Плохо:

```
const user: any = {}
```

Так ты просто отключаешь TS.

Ошибка 3. Пытаются типизировать всё подряд вручную

Плохо:

```
const name: string = 'Dima'
const age: number = 30
const active: boolean = true
```

Если TS и так всё выводит, лучше не перегружать код.

Ошибка 4. Не понимают, что `|` — это “или”

```
let value: string | number
```

Это не “и строка и число одновременно”, а **либо одно, либо другое**.

---

15. Минимальный набор синтаксиса, который ты уже должен запомнить

Вот это — прям база-база:

```
const name: string = 'Dima'
const age: number = 25
const isAdmin: boolean = false

const tags: string[] = ['vue', 'ts']

const user: { id: number; name: string } = {
  id: 1,
  name: 'Dima',
}

let id: string | number = 1

let theme: 'light' | 'dark' = 'light'

let data: unknown = 'hello'
```

Если ты это понимаешь, значит этап 1 уже реально начал заходить.

---

16. Практика

Попробуй сам определить типы для этих переменных.

Задание 1

```
const title = 'TypeScript'
```

Какой тут тип?

Задание 2

```
let count = 10
```

Какой тут тип?

Задание 3

```
const list: ??? = [1, 2, 3]
```

Подставь тип.

Задание 4

```
const user: ??? = {
  id: 1,
  name: 'Alex',
}
```

Подставь тип объекта.

Задание 5

```
let status: ??? = 'loading'
```

Нужно разрешить только:

- `'loading'`
- `'success'`
- `'error'`

Задание 6

```
let value: ??? = null
```

Нужно, чтобы потом можно было записать строку или `null`.

Задание 7

```
const items: ??? = ['a', 1, 'b', 2]
```

Нужно описать массив, где элементы — строка или число.

---

17. Ответы для самопроверки

```
const title = 'TypeScript' // string
```

```
let count = 10 // number
```

```
const list: number[] = [1, 2, 3]
```

```
const user: { id: number; name: string } = {
  id: 1,
  name: 'Alex',
}
```

```
let status: 'loading' | 'success' | 'error' = 'loading'
```

```
let value: string | null = null
```

```
const items: (string | number)[] = ['a', 1, 'b', 2]
```

---

18. Маленькое закрепление на одном примере

Разберём вот этот кусок:

```
const product: {
  id: number
  title: string
  price: number
  category?: string
} = {
  id: 1,
  title: 'Phone',
  price: 50000,
}
```

Что здесь важно:

- `product` — объект
- `id` — число
- `title` — строка
- `price` — число
- `category?` — необязательное поле

То есть объект валиден даже без `category`.

---

19. Что тебе надо уметь после этого этапа

После этапа 1 ты должен уверенно понимать:

- как пишется аннотация типа;
- какие есть базовые типы;
- как типизировать массив;
- как типизировать простой объект;
- что такое union;
- что такое literal type;
- почему `any` опасен;
- чем `unknown` лучше `any`;
- что `null` и `undefined` нужно учитывать явно.

---

20. Что делать прямо сейчас

Вот лучший порядок закрепления:

1. перепиши все примеры руками;
2. сам измени типы и посмотри, где TS ругается;
3. сделай 10 своих примеров:
    - строка,
    - число,
    - boolean,
    - массив строк,
    - массив чисел,
    - объект,
    - объект с необязательным полем,
    - union,
    - literal type,
    - `unknown`.

---

21. Мини-конспект в сжатом виде

```
const name: string = 'Dima'
const age: number = 25
const isAdmin: boolean = true

const numbers: number[] = [1, 2, 3]
const names: string[] = ['a', 'b']

const user: { id: number; name: string; phone?: string } = {
  id: 1,
  name: 'Dima',
}

let id: string | number = 123
let theme: 'light' | 'dark' = 'light'
let value: string | null = null

let data: unknown = 'hello'
```

Дальше логично перейти к этапу 2: **объекты, функции и базовая типизация поведения**, потому что именно там TypeScript начинает ощущаться как рабочий инструмент, а не просто набор подписей.

Пользователь

Этап 2. Типизация объектов, функций, массивов

ChatGPT

Отлично. Этап 2 — это уже настоящая рабочая база.
Именно здесь TypeScript начинает быть полезным в реальном коде.

На этом этапе цель такая:

научиться уверенно типизировать

- объекты,
- массивы,
- функции,
- массивы объектов,
- функции, которые принимают объекты и возвращают объекты.

Это фундамент почти всего прикладного TS.

Этап 2. Типизация объектов, функций, массивов

1. Типизация объектов

На прошлом этапе ты уже видел простые объекты:

```
const user: { id: number; name: string } = {
  id: 1,
  name: 'Dima',
}
```

Это рабочий вариант, но важно понять логику.

Запись

```
{ id: number; name: string }
```

означает:

- у объекта есть поле `id`, оно должно быть `number`
- у объекта есть поле `name`, оно должно быть `string`

Если не хватает поля — ошибка.
Если тип поля не совпадает — ошибка.

Пример ошибки:

```
const user: { id: number; name: string } = {
  id: '1',
  name: 'Dima',
}
```

Потому что `id` должен быть числом, а не строкой.

---

2. Необязательные поля в объектах

```
const user: { id: number; name: string; phone?: string } = {
  id: 1,
  name: 'Dima',
}
```

`phone?: string` означает:

- поле может быть строкой,
- а может вообще отсутствовать.

Если поле есть, оно должно соответствовать типу:

```
const user: { id: number; name: string; phone?: string } = {
  id: 1,
  name: 'Dima',
  phone: '+79999999999',
}
```

Если написать так:

```
const user: { id: number; name: string; phone?: string } = {
  id: 1,
  name: 'Dima',
  phone: 123,
}
```

будет ошибка.

---

3. Вложенные объекты

Часто объект содержит другой объект.

```
const user: {
  id: number
  name: string
  address: {
    city: string
    zip: number
  }
} = {
  id: 1,
  name: 'Dima',
  address: {
    city: 'Moscow',
    zip: 123456,
  },
}
```

Здесь TypeScript проверяет всю структуру целиком.

Если внутри `address` что-то не так, он тоже покажет ошибку.

---

4. Массивы

Простейший случай:

```
const numbers: number[] = [1, 2, 3]
const names: string[] = ['Ann', 'Bob']
```

Если массив должен содержать только строки, число туда уже не положишь:

```
const names: string[] = ['Ann', 'Bob']
names.push(123) // ошибка
```

---

5. Массив объектов

Это одна из самых частых конструкций в реальном коде.

```
const users: { id: number; name: string }[] = [
  { id: 1, name: 'Ann' },
  { id: 2, name: 'Bob' },
]
```

Читается так:

`users` — это массив объектов,
где каждый объект имеет:

- `id: number`
- `name: string`

Если один объект в массиве не соответствует структуре, будет ошибка.

```
const users: { id: number; name: string }[] = [
  { id: 1, name: 'Ann' },
  { id: 2, name: 123 }, // ошибка
]
```

---

6. Когда inline-тип объекта становится неудобным

Вот так писать можно:

```
const users: { id: number; name: string; email?: string }[] = [
  { id: 1, name: 'Ann' },
  { id: 2, name: 'Bob', email: 'bob@test.com' },
]
```

Но когда структура повторяется, лучше вынести её в отдельный тип:

```
type User = {
  id: number
  name: string
  email?: string
}

const users: User[] = [
  { id: 1, name: 'Ann' },
  { id: 2, name: 'Bob', email: 'bob@test.com' },
]
```

Это намного удобнее и чище.

---

7. Типизация функций

Теперь самое важное: функции.

Базовая форма

```
function sum(a: number, b: number): number {
  return a + b
}
```

Что здесь происходит:

- `a: number` — первый параметр число
- `b: number` — второй параметр число
- `: number` после скобок — функция должна вернуть число

Стрелочная функция

```
const sum = (a: number, b: number): number => {
  return a + b
}
```

То же самое, просто другой синтаксис.

---

8. Если функция ничего не возвращает

```
function logMessage(message: string): void {
  console.log(message)
}
```

`void` означает: полезного возвращаемого значения нет.

Такие функции что-то делают, но ничего не возвращают для дальнейшего использования.

---

9. Необязательные параметры функции

```
function greet(name: string, age?: number): string {
  if (age !== undefined) {
    return `Hello, ${name}. Age: ${age}`
  }

  return `Hello, ${name}`
}
```

`age?: number` значит, что параметр можно не передавать.

Можно вызвать так:

```
greet('Dima')
greet('Dima', 25)
```

---

10. Значения по умолчанию

```
function createUser(name: string, role: string = 'user'): string {
  return `${name} - ${role}`
}
```

Если `role` не передан, возьмётся `'user'`.

---

11. Функция с объектом в параметре

Это уже очень реальный кейс.

```
function printUser(user: { id: number; name: string }): void {
  console.log(user.name)
}
```

Вызов:

```
printUser({ id: 1, name: 'Dima' })
```

Если не хватает поля — ошибка:

```
printUser({ id: 1 }) // ошибка
```

---

12. Лучше выносить тип объекта в отдельный `type`

```
type User = {
  id: number
  name: string
  email?: string
}

function printUser(user: User): void {
  console.log(user.name)
}
```

Так код становится намного понятнее.

---

13. Функция, возвращающая объект

```
type User = {
  id: number
  name: string
}

function createUser(id: number, name: string): User {
  return {
    id,
    name,
  }
}
```

TypeScript проверит, что функция реально возвращает объект типа `User`.

---

14. Функция, возвращающая массив

```
function getNumbers(): number[] {
  return [1, 2, 3]
}
```

Или массив объектов:

```
type User = {
  id: number
  name: string
}

function getUsers(): User[] {
  return [
    { id: 1, name: 'Ann' },
    { id: 2, name: 'Bob' },
  ]
}
```

---

15. Функция внутри объекта

Иногда в объекте есть метод.

```
const calculator: {
  sum: (a: number, b: number) => number
} = {
  sum: (a, b) => a + b,
}
```

Здесь поле `sum` — это функция, которая:

- принимает два числа,
- возвращает число.

---

16. Типизация callback-функций

Это тоже очень важно.

```
function processNumbers(
  numbers: number[],
  callback: (item: number) => number
): number[] {
  return numbers.map(callback)
}
```

Здесь:

- `numbers` — массив чисел
- `callback` — функция, которая принимает `number` и возвращает `number`
- результат всей функции — `number[]`

Пример вызова:

```
const result = processNumbers([1, 2, 3], (item) => item * 2)
```

---

17. Массивы и методы массива

TS хорошо понимает типы в `map`, `filter`, `find`.

Пример:

```
const users: { id: number; name: string }[] = [
  { id: 1, name: 'Ann' },
  { id: 2, name: 'Bob' },
]
```

`map`

```
const names = users.map((user) => user.name)
```

TypeScript поймёт, что `names` — это `string[]`.

`filter`

```
const filtered = users.filter((user) => user.id > 1)
```

`filtered` останется массивом тех же объектов.

---

18. Объектный тип и лишние поля

Смотри:

```
type User = {
  id: number
  name: string
}
```

Если ты передаёшь литерал напрямую, TS проверяет лишние поля строже:

```
function printUser(user: User) {
  console.log(user.name)
}

printUser({
  id: 1,
  name: 'Dima',
  age: 25,
})
```

Здесь TS может ругнуться на `age`, потому что его нет в `User`.

Это полезно: он защищает контракт данных.

---

19. Разница между “объект с полем” и “массив объектов”

Нужно очень чётко различать:

Объект:

```
const user: { id: number; name: string } = {
  id: 1,
  name: 'Dima',
}
```

Массив объектов:

```
const users: { id: number; name: string }[] = [
  { id: 1, name: 'Dima' },
  { id: 2, name: 'Ann' },
]
```

Частая ошибка новичка — забыть `[]`.

---

20. Практический шаблон, который надо запомнить

Очень типичный код:

```
type Product = {
  id: number
  title: string
  price: number
  inStock: boolean
}

const products: Product[] = [
  { id: 1, title: 'Phone', price: 50000, inStock: true },
  { id: 2, title: 'Laptop', price: 90000, inStock: false },
]

function getProductTitles(items: Product[]): string[] {
  return items.map((item) => item.title)
}

function findAvailableProducts(items: Product[]): Product[] {
  return items.filter((item) => item.inStock)
}
```

Если ты понимаешь этот блок полностью — этап идёт правильно.

---

21. Что нужно особенно запомнить на этом этапе

Самые важные формы:

Объект

```
type User = {
  id: number
  name: string
}
```

Массив

```
const users: User[] = []
```

Функция с параметрами

```
function sum(a: number, b: number): number {
  return a + b
}
```

Функция с объектом

```
function printUser(user: User): void {
  console.log(user.name)
}
```

Функция, возвращающая массив

```
function getUsers(): User[] {
  return []
}
```

---

22. Частые ошибки новичка

Ошибка 1. Не типизируют параметры функции

Плохо:

```
function sum(a, b) {
  return a + b
}
```

Правильно:

```
function sum(a: number, b: number): number {
  return a + b
}
```

Ошибка 2. Путают объект и массив объектов

Плохо:

```
const users: { id: number; name: string } = [
  { id: 1, name: 'Ann' },
]
```

Правильно:

```
const users: { id: number; name: string }[] = [
  { id: 1, name: 'Ann' },
]
```

Ошибка 3. Везде пишут огромные inline-типы

Плохо:

```
function printUser(user: { id: number; name: string; email?: string; age?: number }) {
  console.log(user.name)
}
```

Лучше:

```
type User = {
  id: number
  name: string
  email?: string
  age?: number
}

function printUser(user: User) {
  console.log(user.name)
}
```

Ошибка 4. Не указывают возвращаемый тип там, где функция важная

TS часто сам выводит его, но для важных функций полезно писать явно:

```
function createUser(id: number, name: string): User {
  return { id, name }
}
```

---

23. Практика

Теперь задания.

Задание 1

Опиши тип объекта `Car`, где есть:

- `id` — number
- `brand` — string
- `year` — number
- `color` — необязательное string-поле

Задание 2

Создай массив `cars` типа `Car[]` с двумя объектами.

Задание 3

Напиши функцию `getCarBrands`, которая:

- принимает массив машин,
- возвращает массив брендов (`string[]`).

Задание 4

Напиши функцию `createCar`, которая:

- принимает `id`, `brand`, `year`,
- возвращает объект типа `Car`.

Задание 5

Напиши функцию `printCar`, которая:

- принимает объект `Car`,
- ничего не возвращает,
- выводит в консоль бренд и год.

---

24. Решение

```
type Car = {
  id: number
  brand: string
  year: number
  color?: string
}

const cars: Car[] = [
  { id: 1, brand: 'Toyota', year: 2020 },
  { id: 2, brand: 'BMW', year: 2022, color: 'black' },
]

function getCarBrands(items: Car[]): string[] {
  return items.map((item) => item.brand)
}

function createCar(id: number, brand: string, year: number): Car {
  return {
    id,
    brand,
    year,
  }
}

function printCar(car: Car): void {
  console.log(`${car.brand} - ${car.year}`)
}
```

---

25. Мини-конспект этапа

Вот сжатая выжимка, которую стоит сохранить:

```
type User = {
  id: number
  name: string
  email?: string
}

const users: User[] = [
  { id: 1, name: 'Ann' },
  { id: 2, name: 'Bob', email: 'bob@test.com' },
]

function getUserNames(items: User[]): string[] {
  return items.map((item) => item.name)
}

function createUser(id: number, name: string): User {
  return {
    id,
    name,
  }
}

function printUser(user: User): void {
  console.log(user.name)
}
```

---

26. Что ты должен уметь после этапа 2

После этого этапа ты уже должен уверенно уметь:

- описывать тип простого объекта;
- делать необязательные поля;
- типизировать массивы;
- типизировать массивы объектов;
- писать типы параметров функции;
- писать тип возвращаемого значения;
- передавать объект в функцию;
- возвращать объект из функции;
- работать с `map` на типизированных массивах.

Это уже реальная рабочая база.

Дальше логично идти в **Этап 3: union types, narrowing, optional, unknown, null**, потому что именно там начинается безопасная работа с неоднозначными данными и условиями.

Пользователь

Этап 3. Union, narrowing, optional, unknown

ChatGPT

Отлично. Это один из самых важных этапов во всём TypeScript.

Потому что в реальном коде данные почти никогда не бывают “идеально простыми”. Очень часто значение может быть:

- либо строкой, либо числом;
- либо объектом, либо `null`;
- либо ещё не загружено;
- либо пришло из API в непонятной форме;
- либо поле вообще отсутствует.

Именно для этого нужны:

- `union`
- `narrowing`
- optional-поля
- `unknown`

Этап 3. Union, narrowing, optional, unknown

1. Union type — значение может быть разного типа

Union записывается через `|`.

```
let value: string | number

value = 'hello'
value = 123
```

Это читается так:

`value` может быть **либо** строкой, **либо** числом.

Важно: это не значит, что значение одновременно и строка, и число.
Это значит: в конкретный момент времени там только один из допустимых типов.

---

2. Где union встречается постоянно

Строка или `null`

```
let selectedUser: string | null = null
selectedUser = 'Dima'
selectedUser = null
```

Очень частая ситуация во фронте:

- сначала ничего не выбрано;
- потом выбрали пользователя.

Число или строка

```
let id: number | string

id = 10
id = '10'
```

Такое часто встречается в параметрах, route params, API.

Объект или `undefined`

```
type User = {
  id: number
  name: string
}

let currentUser: User | undefined = undefined
```

Например:

- пользователь ещё не загружен;
- или не найден.

---

3. Почему union сам по себе недостаточен

Вот пример:

```
function printValue(value: string | number) {
  console.log(value.toUpperCase())
}
```

Здесь ошибка.

Почему?
Потому что `toUpperCase()` есть у строки, но нет у числа.

TypeScript говорит:
“Ты объявил, что это может быть `string | number`. Я не дам вызывать строковый метод без проверки”.

И это правильно.

---

4. Narrowing — сужение типа

Narrowing — это когда ты проверкой доказываешь TypeScript, какой тип у значения сейчас.

После проверки TS “сужает” тип.

4.1. Narrowing через `typeof`

```
function printValue(value: string | number) {
  if (typeof value === 'string') {
    console.log(value.toUpperCase())
  } else {
    console.log(value.toFixed(2))
  }
}
```

Что происходит:

- в ветке `if` TypeScript понимает: `value` — `string`
- в ветке `else` TypeScript понимает: `value` — `number`

Это базовый и очень важный способ narrowing.

4.2. Narrowing через проверку на `null`

```
function printName(name: string | null) {
  if (name !== null) {
    console.log(name.toUpperCase())
  }
}
```

До проверки `name` — `string | null`
После `name !== null` — просто `string`

4.3. Narrowing через `undefined`

```
function printUserName(user: { name: string } | undefined) {
  if (user !== undefined) {
    console.log(user.name)
  }
}
```

4.4. Narrowing через `in`

Когда есть union объектов.

```
type User = {
  name: string
}

type Admin = {
  name: string
  permissions: string[]
}

function printPerson(person: User | Admin) {
  if ('permissions' in person) {
    console.log(person.permissions.join(', '))
  } else {
    console.log(person.name)
  }
}
```

Здесь:

- если в объекте есть `permissions`, значит это `Admin`
- иначе это `User`

---

5. Optional fields — необязательные поля

Ты уже видел это раньше, но теперь надо понять глубже.

```
type User = {
  id: number
  name: string
  phone?: string
}
```

`phone?: string` означает:

- поля может вообще не быть;
- если поле есть, оно должно быть строкой.

Оба варианта валидны:

```
const user1: User = {
  id: 1,
  name: 'Dima',
}
```

```
const user2: User = {
  id: 2,
  name: 'Ann',
  phone: '+79999999999',
}
```

---

6. Важная разница: `phone?: string` и `phone: string | undefined`

Эти записи похожи, но это не одно и то же.

Вариант 1

```
type User = {
  phone?: string
}
```

Здесь поле может отсутствовать совсем.

Вариант 2

```
type User = {
  phone: string | undefined
}
```

Здесь поле обязательно должно быть в объекте, но значение может быть `undefined`.

Пример:

```
const a: { phone?: string } = {}
```

Это нормально.

А вот здесь:

```
const b: { phone: string | undefined } = {}
```

Будет ошибка, потому что поле `phone` обязательно.

Это очень важное различие.

---

7. Как работать с optional-полями

Нельзя бездумно обращаться к необязательному полю:

```
type User = {
  name: string
  phone?: string
}

function printPhone(user: User) {
  console.log(user.phone.toUpperCase())
}
```

Здесь ошибка, потому что `phone` может не существовать.

Правильно:

```
function printPhone(user: User) {
  if (user.phone !== undefined) {
    console.log(user.phone.toUpperCase())
  }
}
```

Или короче — через optional chaining:

```
function printPhone(user: User) {
  console.log(user.phone?.toUpperCase())
}
```

---

8. Optional chaining — `?.`

Это очень полезный синтаксис.

```
const city = user.address?.city
```

Смысл:

- если `address` есть, возьми `city`
- если `address` нет, верни `undefined`

Без него пришлось бы писать длиннее:

```
const city = user.address ? user.address.city : undefined
```

---

9. Nullish coalescing — `??`

Очень часто идёт рядом с optional chaining.

```
const phone = user.phone ?? 'Не указан'
```

Смысл:

- если `user.phone` равен `null` или `undefined`, подставь `'Не указан'`
- иначе используй настоящее значение

Пример:

```
const title = value ?? 'default'
```

Это лучше, чем `||`, когда тебе важны значения `0`, `false`, `''`.

Например:

```
const count = 0 || 10 // 10
const correctCount = 0 ?? 10 // 0
```

---

10. `unknown` — неизвестный тип

Это очень важный тип для безопасной работы.

```
let value: unknown
```

Он означает:

“Я пока не знаю, что это за значение”.

Например, пришло что-то извне:

- из API,
- из `JSON.parse`,

- из `catch`,
- из сторонней библиотеки.

---

11. Чем `unknown` лучше `any`

`any`

```
let value: any = 'hello'
value.toUpperCase()
value.test().a.b.c
```

TS ничего не проверяет.

`unknown`

```
let value: unknown = 'hello'
value.toUpperCase() // ошибка
```

С `unknown` сначала надо проверить тип:

```
let value: unknown = 'hello'

if (typeof value === 'string') {
  console.log(value.toUpperCase())
}
```

Это делает код безопаснее.

---

12. Когда использовать `unknown`

Очень типичный пример:

```
function handleResponse(data: unknown) {
  if (typeof data === 'string') {
    console.log(data.toUpperCase())
  }
}
```

Или:

```
function logError(error: unknown) {
  if (error instanceof Error) {
    console.log(error.message)
  } else {
    console.log('Unknown error')
  }
}
```

Это нормальная и правильная практика.

---

13. `unknown` в `catch`

Полезная реальная ситуация:

```
try {
  throw new Error('Something went wrong')
} catch (error: unknown) {
  if (error instanceof Error) {
    console.log(error.message)
  } else {
    console.log('Неизвестная ошибка')
  }
}
```

Потому что ошибка в JS вообще не обязана быть объектом `Error`.

---

14. Union объектов

Очень важная тема для прикладного кода.

```
type LoadingState = {
  status: 'loading'
}

type SuccessState = {
  status: 'success'
  data: string[]
}

type ErrorState = {
  status: 'error'
  message: string
}

type RequestState = LoadingState | SuccessState | ErrorState
```

Теперь функция:

```
function handleState(state: RequestState) {
  if (state.status === 'loading') {
    console.log('Loading...')
  }

  if (state.status === 'success') {
    console.log(state.data)
  }

  if (state.status === 'error') {
    console.log(state.message)
  }
}
```

Это очень мощный паттерн.
Он часто используется для состояний загрузки, форм, запросов.

---

15. Почему literal-поле удобно для narrowing

Смотри:

```
type Success = {
  type: 'success'
  data: string
}

type ErrorResult = {
  type: 'error'
  message: string
}

type Result = Success | ErrorResult
```

Теперь TS умеет понимать тип по полю `type`:

```
function printResult(result: Result) {
  if (result.type === 'success') {
    console.log(result.data)
  } else {
    console.log(result.message)
  }
}
```

Это называется **discriminated union**.
Очень полезная модель для хорошего TypeScript.

---

16. Частая ошибка с union

Вот так нельзя:

```
function format(value: string | number) {
  return value.trim()
}
```

Потому что `trim()` есть только у строки.

Нужно сначала сузить тип:

```
function format(value: string | number) {
  if (typeof value === 'string') {
    return value.trim()
  }

  return value.toString()
}
```

---

17. Частая ошибка с optional

Вот так нельзя:

```
type User = {
  phone?: string
}

function test(user: User) {
  return user.phone.length
}
```

Потому что `phone` может отсутствовать.

Правильно:

```
function test(user: User) {
  return user.phone?.length ?? 0
}
```

---

18. Частая ошибка с `unknown`

Вот так нельзя:

```
function test(value: unknown) {
  return value.toUpperCase()
}
```

Нужно проверять:

```
function test(value: unknown) {
  if (typeof value === 'string') {
    return value.toUpperCase()
  }

  return 'Not a string'
}
```

---

19. Практический пример целиком

```
type User = {
  id: number
  name: string
  phone?: string
}

function printUser(user: User | null): void {
  if (user === null) {
    console.log('Пользователь не найден')
    return
  }

  console.log(user.name)
  console.log(user.phone ?? 'Телефон не указан')
}
```

Что здесь используется:

- `User | null`
- narrowing через `user === null`
- optional-поле `phone?`
- `??` для значения по умолчанию

Это уже очень похоже на реальный код.

---

20. Ещё один хороший пример с `unknown`

```
function parseInput(value: unknown): string {
  if (typeof value === 'string') {
    return value.trim()
  }

  if (typeof value === 'number') {
    return value.toString()
  }

  return 'Unsupported value'
}
```

Здесь ты безопасно работаешь с неизвестными данными.

---

21. Практика

Задание 1

Объяви переменную `id`, которая может быть строкой или числом.

Задание 2

Объяви переменную `selectedTab`, которая может быть:

- `'home'`
- `'settings'`
- `'profile'`
- `null`

Задание 3

Создай тип `User`, где:

- `id: number`
- `name: string`
- `email?: string`

Задание 4

Напиши функцию `printEmail`, которая:

- принимает `User`
- если email есть, выводит его в верхнем регистре
- если нет, выводит `'Email not set'`

Задание 5

Напиши функцию `formatValue`, которая принимает `string | number` и:

- если строка — возвращает `trim()`
- если число — возвращает строку через `toString()`

Задание 6

Напиши функцию `handleUnknown`, которая принимает `unknown` и:

- если это строка — возвращает её длину
- иначе возвращает `0`

Задание 7

Создай union-состояние запроса:

- loading
- success с полем `data: string[]`
- error с полем `message: string`

И напиши функцию, которая обрабатывает эти состояния.

---

22. Решение

```
let id: string | number

let selectedTab: 'home' | 'settings' | 'profile' | null = null

type User = {
  id: number
  name: string
  email?: string
}

function printEmail(user: User): void {
  if (user.email !== undefined) {
    console.log(user.email.toUpperCase())
  } else {
    console.log('Email not set')
  }
}

function formatValue(value: string | number): string {
  if (typeof value === 'string') {
    return value.trim()
  }

  return value.toString()
}

function handleUnknown(value: unknown): number {
  if (typeof value === 'string') {
    return value.length
  }

  return 0
}

type LoadingState = {
  status: 'loading'
}

type SuccessState = {
  status: 'success'
  data: string[]
}

type ErrorState = {
  status: 'error'
  message: string
}
```

```
type RequestState = LoadingState | SuccessState | ErrorState

function handleState(state: RequestState): void {
  if (state.status === 'loading') {
    console.log('Loading...')
    return
  }

  if (state.status === 'success') {
    console.log(state.data)
    return
  }

  console.log(state.message)
}
```

---

23. Мини-конспект этапа

Вот ядро этого этапа:

```
let value: string | number
let user: { name: string } | null = null

type User = {
  id: number
  name: string
  phone?: string
}

function printValue(value: string | number) {
  if (typeof value === 'string') {
    return value.toUpperCase()
  }

  return value.toFixed(2)
}

function printPhone(user: User) {
  return user.phone?.toUpperCase() ?? 'No phone'
}

function handleUnknown(value: unknown) {
  if (typeof value === 'string') {
    return value.length
  }

  return 0
}
```

---

24. Что ты должен уметь после этапа 3

После этого этапа ты уже должен уверенно понимать:

- что такое union;
- как работает `|`;
- почему union требует проверок;
- как делать narrowing через `typeof`, `in`, `null`, `undefined`;
- что значит optional-поле;
- разницу между `prop?: T` и `prop: T | undefined`;
- как использовать `?.`;
- как использовать `??`;
- почему `unknown` безопаснее `any`;
- как обрабатывать неоднозначные данные.

Это уже очень серьёзный кусок TypeScript.

Дальше логично идти в **Этап 4: `type`, `interface`, `readonly`, переиспользуемые модели данных**, потому что там начинается нормальная архитектура типов, а не просто отдельные подписи.

Пользователь

сделай методичку по этим этапам Самая частая ошибка — пытаться сразу учить:

generics, infer, utility types, сложные типы Vue, типы библиотек.

Так делать не надо.

Правильный порядок такой:

Этап 1. Основа синтаксиса типов Этап 2. Типизация объектов, функций, массивов Этап 3. Union, narrowing, optional, unknown Этап 4. Interfaces, type aliases, readonly, partial формы Этап 5. Generics Этап 6. Utility types Этап 7. Типизация API, асинхронности, ошибок Этап 8. Чтение сложных типов в реальном коде Этап 9. Vue 3 + Composition API + TS

То есть сначала ты должен начать думать типами, а уже потом переносить это в Vue.

ChatGPT

Deep Research has started preparing a comprehensive, structured TypeScript learning guide based strictly on the staged progression you outlined — from fundamentals to Vue 3 + Composition API integration.

You will receive a detailed methodological document with explanations, examples, exercises, common mistakes, and learning recommendations shortly.

Пользователь

я просил методичку по этапам, разложи мне базу по Этап 1. Основа синтаксиса типов Этап 2. Типизация объектов, функций, массивов Этап 3. Union, narrowing, optional, unknown Этап 4. Interfaces, type aliases, readonly, partial формы Этап 5. Generics Этап 6. Utility types Этап 7. Типизация API, асинхронности, ошибок Этап 8. Чтение сложных типов в реальном коде Этап 9. Vue 3 + Composition API + TS формат: этап 1 - теория, этап 2 - теория и тд

ChatGPT

Deep Research has started preparing a structured, stage-by-stage TypeScript methodology strictly in the format you specified (each stage — theory), without skipping levels or mixing topics.

You will receive the full structured guide shortly.

Пользователь

Этап 4. Interfaces, type aliases, readonly, partial формы

ChatGPT

Deep Research has started preparing a detailed practical breakdown of **Stage 4: Interfaces, type aliases, readonly, and partial forms**, including comparisons, examples, common mistakes, and exercises.

You will receive the structured material shortly.

Пользователь

я просил методичку по этапам, разложи мне базу по Этап 1. Основа синтаксиса типов Этап 2. Типизация объектов, функций, массивов Этап 3. Union, narrowing, optional, unknown Этап 4. Interfaces, type aliases, readonly, partial формы Этап 5. Generics Этап 6. Utility types Этап 7. Типизация API, асинхронности, ошибок Этап 8. Чтение сложных типов в реальном коде Этап 9. Vue 3 + Composition API + TS формат: этап 1 - теория, этап 2 - теория и тд

ChatGPT

Deep Research has started preparing a structured, stage-by-stage TypeScript methodology strictly in the format you specified (each stage — theory), without skipping levels or mixing topics.

You will receive the full structured guide shortly.

Пользователь

Этап 4. Interfaces, type aliases, readonly, partial формы

ChatGPT

Отлично. Тогда идём в нужном формате:

Этап 4. Interfaces, type aliases, readonly, partial формы — теория

На этом этапе ты переходишь от “я умею подписывать типы” к “я умею **строить модели данных**”.

Если на первых этапах ты учился типизировать отдельные переменные, объекты и функции, то здесь ты начинаешь описывать **контракты данных**:

- какие поля у сущности есть всегда;
- какие поля могут отсутствовать;
- какие поля нельзя менять;
- как переиспользовать одну и ту же форму данных в разных местах.

Главная цель этапа: **научиться выносить типы в отдельные модели и управлять их формой.**

1. Зачем вообще нужны `interface` и `type`

До этого можно было писать так:

```
const user: { id: number; name: string; email?: string } = {
  id: 1,
  name: 'Dima',
}
```

Но как только такая структура начинает повторяться в нескольких местах, inline-описание становится неудобным:

- трудно читать;
- трудно переиспользовать;
- трудно менять;
- код быстро захламляется.

Поэтому типы выносят отдельно:

```
type User = {
  id: number
  name: string
  email?: string
}
```

или

```
interface User {
  id: number
  name: string
  email?: string
}
```

Теперь эту форму можно использовать везде:

- в параметрах функций;
- в возвращаемых значениях;
- в массивах;
- в API-ответах;
- в компонентах.

2. `interface`

`interface` — это способ описать структуру объекта.

```
interface User {
  id: number
  name: string
  email?: string
}
```

Это означает:

- у объекта есть `id: number`;
- есть `name: string`;
- поле `email` необязательное.

Использование:

```
const user: User = {
  id: 1,
  name: 'Dima',
}
```

`interface` хорошо подходит, когда ты описываешь:

- объект;
- сущность;
- параметры;
- результат работы;
- форму пропсов;
- форму состояния.

Именно поэтому во многих проектах интерфейсы используют для “основных сущностей”.

3. `type alias`

`type` тоже позволяет описывать структуру объекта:

```
type User = {
  id: number
  name: string
  email?: string
}
```

На этом уровне это выглядит почти так же, как `interface`.

Но `type` шире. Через него можно описывать не только объекты, но и:

- union:

```
type Status = 'idle' | 'loading' | 'success' | 'error'
```

- примитивы:

```
type UserId = number
```

- комбинации типов:

```
type Id = string | number
```

- пересечения:

```
type AdminUser = User & { permissions: string[] }
```

То есть:

- `interface` в первую очередь про форму объекта;
- `type` — более универсальный инструмент.

4. Когда использовать `interface`, а когда `type`

На практике есть два нормальных подхода.

Подход 1

Использовать `interface` для объектов, а `type` для всего остального.

Это очень распространённый стиль:

```
interface User {
  id: number
  name: string
}

type Status = 'loading' | 'success' | 'error'
type Id = string | number
```

Подход 2

Почти всё писать через `type`.

Тоже нормальный подход, особенно в современном TS-коде.

Практический вывод

Для тебя на этом этапе лучше держать такую логику:

- **`interface`** — когда описываешь объектную сущность;
- **`type`** — когда описываешь union, alias, составной тип, пересечение.

5. Главное отличие `interface` и `type`

На базовом уровне они похожи. Но есть важное отличие:

`interface` можно расширять декларативно

```
interface User {
  id: number
  name: string
}

interface User {
  email?: string
}
```

В итоге TypeScript объединит это в одну структуру.

Для `type` так нельзя:

```
type User = {
  id: number
  name: string
}

type User = {
  email?: string
}
```

Это будет ошибкой.

6. Расширение типов

Иногда одна модель строится на основе другой.

Через `interface extends`

```
interface User {
  id: number
  name: string
}

interface Admin extends User {
  permissions: string[]
}
```

Через `type &`

```
type User = {
  id: number
  name: string
}

type Admin = User & {
  permissions: string[]
}
```

7. `readonly`

`readonly` означает: поле существует, но изменять его нельзя.

```
interface User {
  readonly id: number
  name: string
}
```

Использование:

```
const user: User = {
  id: 1,
  name: 'Dima',
}

user.name = 'Alex' // нормально
user.id = 2 // ошибка
```

8. Зачем нужен `readonly`

Он нужен там, где поле не должно меняться после создания объекта:

- `id`;
- `createdAt`;
- системные ключи;
- данные, которые считаются неизменяемыми.

9. `readonly` на массивах

```
const items: readonly string[] = ['a', 'b', 'c']
```

Теперь нельзя:

```
items.push('d') // ошибка
items[0] = 'x' // ошибка
```

10. Поверхностная неизменяемость

Важно понимать: `readonly` защищает только на текущем уровне.

```
interface User {
  readonly id: number
  profile: {
    city: string
  }
}
```

Здесь нельзя переassign `id`, но вложенный объект по-прежнему можно менять:

```
user.profile.city = 'Moscow'
```

11. Partial формы — идея

Очень частый реальный кейс: у тебя есть полная форма объекта, но иногда нужен объект “не целиком”, а только с частью полей.

Например, есть пользователь:

```
interface User {
  id: number
  name: string
  email: string
}
```

Но для обновления ты не всегда отправляешь весь объект. Иногда отправляешь только одно поле:

```
{
  name: 'New Name'
}
```

Вот здесь и появляется идея **partial формы**: **сделать все поля временно необязательными**.

12. `Partial<T>`

`Partial` — это встроенный utility type, но на этом этапе важно понять его именно как форму.

```
interface User {
  id: number
  name: string
  email: string
}

type UserPatch = Partial<User>
```

Теперь `UserPatch` выглядит так по смыслу:

```
type UserPatch = {
  id?: number
  name?: string
  email?: string
}
```

13. Где реально используется `Partial`

Очень часто:

- обновление формы;
- patch-запросы;
- локальные частичные изменения;
- функции, которые принимают только часть настроек;
- объект конфигурации.

Пример:

```
interface Settings {
  theme: string
  language: string
  sidebarOpen: boolean
}

function updateSettings(patch: Partial<Settings>) {
  // принимаем только изменённые поля
}
```

14. “Partial формы” как мышление

На этом этапе тебе важно понять не только `Partial<T>`, а сам принцип:

есть:

- **полная форма сущности**;
- **форма создания**;

- **форма обновления**;
- **форма для отображения**;
- **форма для фильтров**.

Одна и та же сущность в коде почти никогда не существует в одной-единственной форме.

15. Почему нельзя делать всё одним типом

Частая ошибка новичка — пытаться использовать один `User` вообще везде:

- и для API;
- и для формы;
- и для таблицы;
- и для PATCH;
- и для создания.

Так делать не надо.

16. Базовые формы, которые полезно различать

Полная сущность

```
interface User {
  id: number
  name: string
  email: string
}
```

Форма создания

```
type CreateUserDto = {
  name: string
  email: string
}
```

Форма обновления

```
type UpdateUserDto = Partial<CreateUserDto>
```

Упрощённая форма для списка

```
type UserListItem = {
  id: number
  name: string
}
```

17. `readonly` и partial вместе

Иногда у тебя есть поля, которые менять нельзя, и поля, которые можно обновлять частично.

```
interface User {
  readonly id: number
  name: string
  email: string
}
```

Полный объект пользователя имеет неизменяемый `id`, но форма обновления может быть такой:

```
type UpdateUserDto = Partial<Pick<User, 'name' | 'email'>>
```

18. Частые ошибки на этом этапе

Ошибка 1. Не понимают, что `interface` и `type` — это модели, а не значения

Нельзя так:

```
interface User {
  id: number
}
```

и думать, что `User` — это объект.
`User` — это описание формы, а не само значение.

Ошибка 2. Используют один огромный тип для всех сценариев

Ошибка 3. Путают `readonly` и неизменяемость всего объекта

Ошибка 4. Не различают “необязательное поле” и “частичная форма”

19. Как мыслить правильно на этом этапе

Нужно перейти к таким вопросам:

- это основная сущность или временная форма?
- какое поле обязательно всегда?
- какое поле может отсутствовать?
- какое поле нельзя менять?
- один и тот же тип мне правда нужен везде, или нужны разные формы?

20. Практически полезный шаблон мышления

Когда создаёшь тип, спрашивай себя:

1. Это полная сущность или DTO?
2. Какие поля обязательные?
3. Какие поля необязательные?
4. Есть ли поля только для чтения?
5. Нужна ли частичная версия для обновления?
6. Это лучше выразить через `interface` или через `type`?

21. Что нужно вынести из этапа 4

После этого этапа ты должен понимать:

- `interface` — это способ описывать форму объекта;
- `type` — более универсальный инструмент для алиасов, union и композиций;
- оба инструмента можно использовать для объектных моделей;
- `readonly` защищает поля от изменения;
- partial формы нужны для неполных объектов;
- одна сущность в коде может иметь несколько разных форм;
- типы надо строить под сценарий, а не “один тип на всё”.

22. Мини-конспект этапа

```
interface User {
  readonly id: number
  name: string
  email?: string
}

type Status = 'idle' | 'loading' | 'success' | 'error'

type Admin = User & {
  permissions: string[]
}

type UserPatch = Partial<User>
```

23. Связь с дальнейшими этапами

Этот этап нужен, чтобы дальше нормально понять:

- **Этап 5. Generics**;
- **Этап 6. Utility types**;

- **Этап 7. API и async**;
- **Этап 9. Vue 3 + TS**.

Если хочешь, следующим сообщением я разложу в таком же формате **Этап 5. Generics — теория**.

Пользователь

Этап 5. Generics

ChatGPT

Отлично. Продолжаем в том же формате.

Этап 5. Generics — теория

На этом этапе ты переходишь к одной из самых важных идей TypeScript: **писать типы не под одно конкретное значение, а под целый класс значений.**

До этого ты в основном работал так:

- здесь строка;
- здесь число;
- здесь объект `User`;
- здесь массив `User[]`.

Но в реальном коде очень часто нужна **универсальная логика**, которая работает не только с `User`, не только со `string`, а вообще с “любыми данными при сохранении информации об их типе”.

Именно это и решают **generics**.

1. Что такое generics простыми словами

Generics — это параметризованные типы.

То есть ты пишешь не: “эта функция работает только со строкой”

а: “эта функция работает с любым типом, и TypeScript должен сохранить этот тип”.

Самая простая идея:

```
function identity<T>(value: T): T {
  return value
}
```

2. Почему без generics неудобно

Без generics можно сделать так:

```
function identity(value: any): any {
  return value
}
```

Но есть проблема: `any` уничтожает информацию о типе.

3. Чем generics лучше `any`

```
function identity<T>(value: T): T {
  return value
}

const result = identity('hello')
```

Теперь `result` будет `string`, а не `any`.

4. Как читать запись `<T>`

`T` — не магическое слово. Это просто имя.

5. Первая интуиция о generic

Generic нужен, когда ты хочешь сказать:

“Я пока не знаю конкретный тип, но хочу, чтобы TypeScript подставил его позже и сохранил связь между частями конструкции.”

6. Самая важная мысль про generics

Generics нужны не просто для “гибкости”.

Они нужны для того, чтобы **связать типы между собой**.

7. Generic-функции

```
function getLastItem<T>(items: T[]): T | undefined {
  return items[items.length - 1]
}
```

8. Type inference — почему часто не нужно писать `<T>` руками

Обычно TypeScript сам выводит generic-тип.

9. Когда generic можно указать явно

Иногда полезно подставить тип вручную.

10. Generic у стрелочных функций

```
const identity = <T>(value: T): T => {
  return value
}
```

11. Generics для массивов и коллекций

```
function mapToNames<T extends { name: string }>(items: T[]): string[] {
  return items.map(item => item.name)
}
```

12. Generic-типы

```
type ApiResponse<T> = {
  data: T
  error: string | null
  success: boolean
}
```

13. Почему generic-типы так полезны

В реальном проекте у тебя часто повторяется одна и та же оболочка, но с разным содержимым.

14. Generic-interface

```
interface ApiResponse<T> {
  data: T
  error: string | null
  success: boolean
}
```

15. Generic-классы и generic-интерфейсы — общая идея

Generics можно навешивать почти на любую конструкцию, где есть повторяемый шаблон с переменным типом внутри.

16. Несколько generic-параметров

```
function pair<T, U>(first: T, second: U): [T, U] {
  return [first, second]
}
```

17. Более жизненный пример с несколькими параметрами

```
type KeyValuePair<K, V> = {
  key: K
  value: V
}
```

18. Generic constraints — ограничения generic-типа

```
function getId<T extends { id: number }>(item: T): number {
  return item.id
}
```

19. Зачем нужны ограничения

Без ограничения TypeScript не даст обратиться к полю, потому что не знает, что оно существует.

20. Как понимать `extends` в generics

На этом этапе `extends` лучше читать так:

“T должен соответствовать такому условию”

21. Generic и повторное использование логики

Generics позволяют заменить несколько однотипных функций одной универсальной.

22. Generics в реальных проектах

На практике generics постоянно встречаются в таких местах:

- API-обёртки;
- таблицы;
- формы;
- хуки;

- утилиты для массивов;
- состояния `data/loading/error`;
- `Promise<T>`;
- `Record<K, V>`.

23. `Promise<T>` как важнейший реальный пример generic

```
async function loadUser(): Promise<User> {
  return { id: 1, name: 'Dima' }
}
```

24. Generic и формы API-данных

```
type ApiState<T> = {
  data: T | null
  loading: boolean
  error: string | null
}
```

25. Generic и переиспользуемые компоненты

Generics — это архитектурное применение шаблонов к типизации универсальных инструментов.

26. Типовая ошибка новичка: generic ради generic

Generic нужен не везде.

27. Когда generic действительно нужен

Нужен, если:

- функция работает с разными типами;
- результат зависит от входного типа;
- ты делаешь шаблон типа;
- ты строишь переиспользуемую утилиту;
- ты не хочешь терять типизацию через `any`.

28. Частая ошибка: путать generic и union

Union перечисляет допустимые варианты, а generic сохраняет конкретный выбранный вариант.

29. Generic и связь между аргументами

```
function mergeArrays<T>(a: T[], b: T[]): T[] {
  return [...a, ...b]
}
```

30. Generic может быть вложенным

```
type PaginatedResponse<T> = {
  items: T[]
  total: number
  page: number
  limit: number
}
```

31. Значение имён generic-параметров

Имя generic-параметра должно помогать читать код.

32. Как generics меняют мышление

Ты начинаешь проектировать не только значения, но и **семейства типизированных конструкций**.

33. Самые частые ошибки на этом этапе

Ошибка 1. Использовать `any` там, где нужен generic

Ошибка 2. Добавлять `<T>` без реальной универсальности

Ошибка 3. Не ставить ограничения, когда нужен доступ к свойствам

Ошибка 4. Путать generic и конкретный тип

Ошибка 5. Не понимать, что generic сохраняет конкретику

34. Практический шаблон мышления

Когда видишь задачу, спрашивай себя:

1. Эта логика работает только для одного типа или для многих?
2. Нужно ли сохранить связь между аргументом и результатом?
3. Теряю ли я типизацию, если пишу `any`?
4. Есть ли требования к форме данных?

5. Нужен ли `extends`?
6. Это лучше выразить generic-функцией или generic-типом?

35. Что особенно важно для тебя как разработчика

Generics нужны для API-ответов, обобщённых DTO-оболочек, таблиц, хелперов, `Promise`, состояний загрузки и универсальных компонент.

36. Мини-конспект этапа

```
function identity<T>(value: T): T {
  return value
}

function firstItem<T>(items: T[]): T | undefined {
  return items[0]
}

function getId<T extends { id: number }>(item: T): number {
  return item.id
}

type ApiResponse<T> = {
  data: T
  error: string | null
}

type PaginatedResponse<T> = {
  items: T[]
  total: number
}
```

37. Что нужно вынести из этапа 5

После этого этапа ты должен понимать:

- generics — это параметризованные типы;
- generic позволяет писать универсальный код без потери типизации;
- generic сохраняет связь между входом и выходом;
- `extends` задаёт ограничение на допустимую форму;
- generic — это альтернатива плохому `any`, а не украшение кода.

38. Связь с дальнейшими этапами

Этот этап критически важен, потому что дальше без него будет тяжело понять:

- **Этап 6. Utility types**;
- **Этап 7. Типизация API**;
- **Этап 8. Продвинутая типизация**;
- **Этап 9. React/Vue + TS**.

39. Самая короткая суть этапа

Generics — это способ сказать:

“Эта конструкция универсальна, но TypeScript должен помнить конкретный тип, с которым её используют.”

Именно в этом месте TypeScript начинает ощущаться не просто как “аннотации к JavaScript”, а как нормальная система моделирования данных и логики.

Пользователь

Этап 6. Utility types

ChatGPT

Отлично. Идём дальше.

Этап 6. Utility types — теория

На этом этапе ты переходишь от “умения писать свои типы” к следующему важному уровню:

умению преобразовывать уже существующие типы.

Это одна из самых сильных сторон TypeScript.

1. Что такое utility types простыми словами

Utility types — это встроенные служебные типы TypeScript, которые помогают **создавать новые типы на основе других типов**.

2. Почему utility types так важны

В реальной разработке почти никогда не бывает так, что каждый тип существует изолированно.

3. Связь с generics

Почти все utility types построены именно на generics.

4. Главные utility types, которые нужно знать в первую очередь

- `Partial<T>`
- `Required<T>`
- `Readonly<T>`
- `Pick<T, K>`
- `Omit<T, K>`
- `Record<K, V>`
- `Exclude<T, U>`
- `Extract<T, U>`
- `NonNullable<T>`
- `Parameters<T>`
- `ReturnType<T>`
- `Awaited<T>`

Часть 1. `Partial<T>`

5. Что делает `Partial<T>`

`Partial<T>` делает **все свойства типа необязательными**.

6. Когда нужен `Partial`

Это один из самых частых utility types в реальной разработке.

7. Где `Partial` особенно полезен

Очень часто:

- в `PATCH` запросах;
- в формах редактирования;
- в merge/update функциях;
- во временных объектах состояния;
- в конфигурациях с необязательными параметрами.

8. Важный нюанс `Partial`

`Partial<T>` делает поля необязательными только **на верхнем уровне**.

Часть 2. `Required<T>`

9. Что делает `Required<T>`

`Required<T>` — противоположность `Partial<T>`.

10. Когда нужен `Required`

Нужен после валидации или перехода из черновика в полную структуру.

Часть 3. `Readonly<T>`

11. Что делает `Readonly<T>`

`Readonly<T>` делает все свойства типа доступными только для чтения.

12. Что это означает на практике

```
const user: Readonly<User> = {
  id: 1,
  name: 'Dima',
}

user.name = 'Alex' // ошибка
```

13. Когда нужен `Readonly`

Очень полезно:

- для защиты от случайной мутации;
- в immutable-подходе;
- для публичных API;
- для конфигов.

14. Важный нюанс `Readonly`

По умолчанию работает неглубоко.

Часть 4. `Pick<T, K>`

15. Что делает `Pick<T, K>`

Оставляет только выбранные свойства исходного типа.

16. Как читать `Pick<T, K>`

“возьми из `T` только эти поля”.

17. Когда `Pick` особенно полезен

Для DTO, карточек, публичных данных, таблиц, пропсов компонентов.

Часть 5. `Omit<T, K>`

18. Что делает `Omit<T, K>`

Создаёт тип **без некоторых свойств**.

19. Когда `Omit` особенно полезен

Чтобы убрать пароль, системные поля, `id` при создании сущности.

20. `Pick` и `Omit` — как различать

- `Pick` — когда проще перечислить, что **оставить**;
- `Omit` — когда проще перечислить, что **убрать**.

Часть 6. `Record<K, V>`

21. Что делает `Record<K, V>`

Создаёт объектный тип, где ключи имеют тип `K`, значения — тип `V`.

22. Пример с `Record`

```
const scores: Record<string, number> = {
  math: 5,
  physics: 4,
  history: 5,
}
```

23. Где `Record` особенно полезен

Для словарей, маппингов, lookup-таблиц, конфигов и структур “id → сущность”.

24. Очень важный пример `Record`

```
type Status = 'draft' | 'published' | 'archived'

const labels: Record<Status, string> = {
  draft: 'Черновик',
  published: 'Опубликовано',
  archived: 'Архив',
}
```

25. Когда `Record` лучше, чем просто объект

Когда важно не просто сказать “это объект”, а чётко описать ключи и тип значений.

Часть 7. Комбинации utility types

26. Utility types можно комбинировать

```
type EditableUser = Partial<Omit<User, 'id'>>
```

27. Ещё пример комбинации

```
type PublicReadonlyUser = Readonly<Omit<User, 'passwordHash'>>
```

Часть 8. `Exclude<T, U>`

28. Что делает `Exclude<T, U>`

Убирает из union-типа варианты, совместимые с `U`.

29. Как понимать `Exclude`

Это utility type для **вариантов union-типа**, а не для свойств объекта.

30. Когда нужен `Exclude`

Для фильтрации union, исключения состояний и построения подмножеств.

Часть 9. `Extract<T, U>`

31. Что делает `Extract<T, U>`

Оставляет только те варианты union-типа, которые совместимы с `U`.

32. Как различать `Exclude` и `Extract`

- `Exclude` — убрать;
- `Extract` — оставить только нужное.

Часть 10. `NonNullable<T>`

33. Что делает `NonNullable<T>`

Убирает из типа `null` и `undefined`.

34. Когда полезен `NonNullable`

После проверок, для безопасных функций и гарантированно существующих данных.

Часть 11. `Parameters<T>`

35. Что делает `Parameters<T>`

Извлекает типы параметров функции в виде tuple.

36. Зачем нужен `Parameters`

Для обёрток, переиспользования сигнатур и helper-функций.

Часть 12. `ReturnType<T>`

37. Что делает `ReturnType<T>`

Извлекает тип возвращаемого значения функции.

38. Где полезен `ReturnType`

Во фабриках, селекторах, helper-функциях и сервисах.

Часть 13. `Awaited<T>`

39. Что делает `Awaited<T>`

Извлекает тип значения, которое получится после `await`.

40. Почему `Awaited` полезен

В асинхронном коде часто нужен не сам `Promise`, а итоговое значение.

Часть 14. Самые практичные реальные сценарии

41. DTO для создания и редактирования

42. Публичная версия модели

43. Словарь объектов по id

44. Частичное обновление настроек

45. Табличные представления

Часть 15. Как utility types меняют мышление

46. До utility types

47. После utility types

Часть 16. Частые ошибки

48. Ошибка 1. Дублировать то, что уже можно выразить через `Pick` или `Omit`

49. Ошибка 2. Использовать `Partial` там, где нужна строгая структура

50. Ошибка 3. Не различать “свойства объекта” и “варианты union-типа”

51. Ошибка 4. Забывать, что utility types обычно неглубокие

52. Ошибка 5. Пытаться через utility types заменить плохую модель

Часть 17. Мини-конспект по основным utility types

```
type User = {
  id: number
  name: string
  email: string
  passwordHash: string
  createdAt: string
}

type PartialUser = Partial<User>
type RequiredUser = Required<PartialUser>
type ReadonlyUser = Readonly<User>

type UserPreview = Pick<User, 'id' | 'name' | 'email'>
type PublicUser = Omit<User, 'passwordHash'>
type CreateUserDto = Omit<User, 'id' | 'createdAt' | 'passwordHash'>
type UpdateUserDto = Partial<CreateUserDto>

type Status = 'draft' | 'published' | 'archived'
type ActiveStatus = Exclude<Status, 'archived'>
type FinalStatus = Extract<Status, 'published' | 'archived'>

type MaybeString = string | null | undefined
type SafeString = NonNullable<MaybeString>

type Labels = Record<Status, string>
```

Часть 18. Что нужно знать особенно хорошо на практике

В первую очередь:

- `Partial`
- `Pick`
- `Omit`
- `Record`
- `Readonly`

Часть 19. Как быстро выбирать нужный utility type

- все поля optional → `Partial`
- все поля required → `Required`
- запретить изменение → `Readonly`
- оставить поля → `Pick`
- убрать поля → `Omit`
- объект “ключ → значение” → `Record`
- убрать варианты из union → `Exclude`
- оставить часть union → `Extract`
- убрать `null` и `undefined` → `NonNullable`
- тип параметров функции → `Parameters`
- тип результата функции → `ReturnType`
- тип результата после `await` → `Awaited`

Часть 20. Самая короткая суть этапа

Utility types — это встроенные инструменты TypeScript для **преобразования типов без ручного дублирования**.

Часть 21. Что нужно вынести из этапа 6

После этого этапа ты должен понимать:

- utility types берут один тип и строят из него другой;
- `Partial`, `Required`, `Readonly` меняют свойства объекта;
- `Pick` и `Omit` позволяют вырезать части типа;
- `Record` строит словарь;
- `Exclude`, `Extract`, `NonNullable` работают с union-типами;
- `Parameters`, `ReturnType`, `Awaited` извлекают типы из функций и async-логики.

Часть 22. Связь со следующим этапом

Теперь следующий логичный шаг — **Этап 7. Типизация API и асинхронного кода**.

Пользователь

Этап 7. Типизация API, асинхронности, ошибок

ChatGPT

Отлично. Идём дальше в том же формате.

Этап 7. Типизация API, асинхронности, ошибок — теория

На этом этапе TypeScript начинает ощущаться уже не как “я умею подписывать объекты”, а как инструмент для **безопасной работы с внешними данными**.

1. Самая важная мысль этапа

TypeScript **не проверяет сервер в runtime**.
Он не делает API “автоматически правильным”.

Если ты написал:

```
type User = {
  id: number
  name: string
}
```

и потом сделал:

```
const user: User = await fetchUser()
```

это не означает, что сервер реально прислал `User`.
Это означает только, что **ты так объявил в коде**.

Типизация API — это контракт в коде, а не гарантия истины от сервера.

2. Что обычно типизируют в API-слое

1. Данные, которые отправляешь
2. Данные, которые получаешь
3. Асинхронные функции
4. Ошибки и состояния

3. Базовая схема мышления для API

Когда смотришь на endpoint, задавай себе вопросы:

1. Что я отправляю?
2. Что мне приходит в ответ?
3. Это полная сущность или DTO?
4. Что может быть `null`, `undefined`, отсутствовать?
5. Что произойдёт при ошибке?
6. Это один объект, список, пагинация или wrapper?
7. Нужна ли дополнительная трансформация данных?

4. Полная сущность и DTO — это не одно и то же

Полная сущность

```
type User = {
  id: number
  name: string
  email: string
  createdAt: string
}
```

Данные для создания

```
type CreateUserDto = {
  name: string
  email: string
}
```

Данные для обновления

```
type UpdateUserDto = Partial<CreateUserDto>
```

5. Типизация ответа API

```
async function getUser(id: number): Promise<User> {
  // ...
}

async function getUsers(): Promise<User[]> {
  // ...
}
```

6. `Promise<T>` — ключевая конструкция этого этапа

`Promise<User>` означает, что после `await` ты получишь `User`.

7. Асинхронная функция и тип результата

Разделяй:

- что возвращает функция → `Promise<User>`
- что лежит после `await` → `User`

8. Обёртка ответа API

```
type ApiResponse<T> = {
  data: T
  success: boolean
  error: string | null
}
```

9. Пагинация

```
type PaginatedResponse<T> = {
  items: T[]
  total: number
  page: number
  limit: number
}
```

10. Параметры запроса тоже надо типизировать

```
type GetUsersParams = {
  page?: number
  limit?: number
  search?: string
}
```

11. Query-параметры и optional поля

Очень часто параметры API необязательные.

12. `fetch` и проблема “сырых данных”

```
const response = await fetch('/api/users')
const data = await response.json()
```

TypeScript сам по себе не знает, что лежит в `data`.

13. Главный принцип безопасной работы с API

Нужно различать:

1. Предполагаемый контракт;
2. Фактические данные.

14. Сырой ответ и доменная модель

```
type UserApi = {
  id: number
  full_name: string
  created_at: string
}

type User = {
  id: number
  fullName: string
  createdAt: Date
}
```

Нужен mapper:

```
function mapUser(apiUser: UserApi): User {
  return {
    id: apiUser.id,
    fullName: apiUser.full_name,
    createdAt: new Date(apiUser.created_at),
  }
}
```

15. Почему маппинг полезен

Чтобы не тащить snake_case по всему приложению, нормализовать даты и отделять контракт сервера от модели клиента.

16. Асинхронные состояния в UI

```
type LoadingState = {
  status: 'loading'
}

type SuccessState<T> = {
  status: 'success'
  data: T
}

type ErrorState = {
  status: 'error'
  message: string
}

type AsyncState<T> = LoadingState | SuccessState<T> | ErrorState
```

17. Почему `AsyncState<T>` сильнее, чем три разрозненных переменных

Потому что невозможны нелогичные комбинации состояния.

18. Narrowing в async-state

```
function handleUsersState(state: AsyncState<User[]>) {
  if (state.status === 'loading') {
    console.log('Loading...')
    return
  }

  if (state.status === 'error') {
    console.log(state.message)
    return
  }

  console.log(state.data)
}
```

19. Ошибки в `try/catch`

Правильно думать так:

```
try {
  // ...
} catch (error: unknown) {
  // сначала проверяем
}
```

20. Правильная обработка `unknown` в catch

```
try {
  // ...
} catch (error: unknown) {
  if (error instanceof Error) {
    console.log(error.message)
  } else {
    console.log('Unknown error')
  }
}
```

21. Ошибка как строка, объект или Error

В JavaScript можно выбросить вообще что угодно.

22. Собственный тип ошибки API

```
type ApiErrorResponse = {
  message: string
  code: string
  details?: Record<string, string[]>
}
```

23. Type guard для ошибок

```
function isApiError(error: unknown): error is ApiErrorResponse {
  return (
    typeof error === 'object' &&
    error !== null &&
    'message' in error &&
    'code' in error
  )
}
```

24. Функция API как контракт

Хорошая API-функция должна отвечать на 3 вопроса:

1. Что принимает?
2. Что возвращает?
3. Как ведёт себя при ошибке?

25. Где лучше располагать типы

- транспортные типы;
- доменная модель;
- DTO.

26. Почему нельзя бездумно типизировать `response.json()` как готовую модель

Это создаёт ложное ощущение безопасности.

27. Типизация `fetch`-функции

```
async function getUsers(): Promise<User[]> {
  const response = await fetch('/api/users')

  if (!response.ok) {
    throw new Error('Failed to load users')
  }

  const data = await response.json()
  return data
}
```

28. Типизация create/update endpoint

```
type User = {
  id: number
  name: string
  email: string
  createdAt: string
}

type CreateUserDto = Omit<User, 'id' | 'createdAt'>
type UpdateUserDto = Partial<CreateUserDto>
```

29. Когда полезен `Result<T>` вместо throw

```
type Result<T> =
  | { ok: true; data: T }
  | { ok: false; error: string }
```

30. Почему `Result<T>` полезен

Он заставляет явно обработать оба случая.

31. Сервер может прислать `null`

```
async function getCurrentUser(): Promise<User | null> {
  // ...
}
```

32. Необязательные и nullable поля в API

Важно различать:

- поле может отсутствовать;
- поле приходит как `null`.

33. Асинхронные функции и `void`

```
async function deleteUser(id: number): Promise<void> {
  // ...
}
```

34. `Awaited<T>` как полезная модель мышления

Важно понимать разницу между типом promise и итоговым типом после ожидания.

35. Типизация сервисного слоя

```
async function getUsers(params: GetUsersParams): Promise<PaginatedResponse<User>> {
  // ...
}

async function getUser(id: number): Promise<User> {
  // ...
}

async function createUser(payload: CreateUserDto): Promise<User> {
  // ...
}

async function updateUser(id: number, payload: UpdateUserDto): Promise<User> {
  // ...
}

async function deleteUser(id: number): Promise<void> {
  // ...
}
```

36. Как этап 7 связан с предыдущими этапами

Это место, где начинают работать вместе почти все прошлые темы.

37. Частые ошибки на этом этапе

Ошибка 1. Считать, что TS проверяет сервер

Ошибка 2. Использовать один тип на всё

Ошибка 3. Писать `catch (error: any)`

Ошибка 4. Игнорировать `null` и optional поля

Ошибка 5. Не отделять транспортную модель от доменной

Ошибка 6. Возвращать слишком расплывчатые типы

38. Как мыслить правильно на этом этапе

Задавай себе вопросы:

- что именно сервер присылает?
- что именно мне нужно в приложении?

- совпадают ли эти формы?
- где нужно преобразование?
- какие поля могут отсутствовать?
- ошибка будет через `throw` или через `Result<T>`?
- нужно ли типизировать loading/error state отдельно?

39. Мини-конспект этапа

```
type User = {
  id: number
  name: string
  email: string
}

type CreateUserDto = {
  name: string
  email: string
}

type UpdateUserDto = Partial<CreateUserDto>

type ApiResponse<T> = {
  data: T
  success: boolean
  error: string | null
}

type PaginatedResponse<T> = {
  items: T[]
  total: number
  page: number
  limit: number
}

type AsyncState<T> =
  | { status: 'loading' }
  | { status: 'success'; data: T }
  | { status: 'error'; message: string }

type Result<T> =
  | { ok: true; data: T }
  | { ok: false; error: string }
```

40. Что нужно вынести из этапа 7

После этого этапа ты должен понимать:

- как типизировать async-функции через `Promise<T>`;
- почему DTO и сущность — не одно и то же;
- как описывать API-обёртки типа `ApiResponse<T>`;
- как описывать пагинацию;
- как моделировать loading/success/error состояния;
- почему `catch` лучше типизировать через `unknown`;
- почему TS не валидирует серверные данные автоматически;
- зачем иногда нужен mapper между API-моделью и моделью приложения;
- как делать create/update/delete сигнатуры типобезопасными.

41. Самая короткая суть этапа

Этап 7 учит тебя вот чему:

внешние данные ненадёжны, приходят асинхронно, могут ломаться, и TypeScript должен помогать тебе описывать этот поток максимально честно и точно.

Пользователь

Этап 8. Чтение сложных типов в реальном коде

ChatGPT

Отлично. Это очень важный этап, потому что именно здесь TypeScript перестаёт быть “набором отдельных тем” и превращается в навык чтения реального кода.

Этап 8. Чтение сложных типов в реальном коде — теория

На предыдущих этапах ты учился:

- писать базовые типы;
- строить модели;
- использовать generics;
- применять utility types;
- типизировать API и async.

Но в живом проекте проблема часто уже не в том, чтобы **написать** тип, а в том, чтобы **понять**, что вообще означает чужая сигнатура.

Главная цель такая:

научиться разбирать сложные типы по слоям, не пугаться длинных записей и видеть за ними обычную логику.

1. Главная проблема реального TypeScript-кода

В реальном коде ты встречаешь что-то вроде:

```
type ApiResult<T> = Promise<
  | { ok: true; data: T }
  | { ok: false; error: string }
>
```

или:

```
type UpdateUserPayload = Partial<Omit<User, 'id' | 'createdAt'>>
```

или:

```
function useEntity<T extends { id: number }>(items: Ref<T[]>): ComputedRef<T | undefined>
```

2. Главный принцип чтения сложных типов

Сложный тип нужно читать **снаружи внутрь**, по частям.

3. Первый базовый приём: найти “скелет” типа

```
type UpdateUserDto = Partial<Omit<User, 'id' | 'createdAt'>>
```

Читаем по шагам:

1. взять `User`;
2. убрать `id` и `createdAt`;
3. у оставшихся полей сделать `?`.

4. Второй приём: читать тип как цепочку преобразований

```
type PublicUser = Readonly<Omit<User, 'passwordHash'>>
```

5. Третий приём: сначала найти базовую сущность

Если базовый тип неясен, вся надстройка тоже будет непонятной.

6. Четвёртый приём: union нужно читать как варианты, а не как кашу

```
type Result<T> =
  | { ok: true; data: T }
  | { ok: false; error: string }
```

7. Пятый приём: discriminated union надо читать через поле-разделитель

```
type AsyncState<T> =
  | { status: 'loading' }
  | { status: 'success'; data: T }
  | { status: 'error'; message: string }
```

8. Шестой приём: generic читать как “внутрь будет подставлен конкретный тип”

```
type ApiResponse<T> = {
  data: T
  success: boolean
  error: string | null
}
```

9. Седьмой приём: ограничения generic читать как условие

```
function getId<T extends { id: number }>(item: T): number
```

10. Восьмой приём: функция сначала читается как обычная функция, потом уже типами

```
function mapItems<T, R>(items: T[], mapper: (item: T) => R): R[]
```

11. Девятый приём: длинную сигнатуру раскладывать на именованные части

12. Десятый приём: если сложно, временно подставляй конкретный тип вместо `T`

13. Как читать вложенные utility types

```
type EditableUser = Partial<Pick<User, 'name' | 'email'>>
```

14. Как читать типы функций, которые принимают callback

```
function processUsers(
  users: User[],
  handler: (user: User) => string
): string[]
```

15. Как читать `Promise<T>`

```
async function getUsers(): Promise<User[]>
```

16. Как читать `ReturnType<typeof fn>`

```
type UserResult = ReturnType<typeof createUser>
```

17. Как читать `Parameters<typeof fn>`

```
type CreateUserArgs = Parameters<typeof createUser>
```

18. Как читать `Awaited<ReturnType<...>>`

```
type LoadedUsers = Awaited<ReturnType<typeof getUsers>>
```

19. Как читать `Record<K, V>`

```
type UserMap = Record<number, User>
```

20. Как читать “объект по ключам типа”

```
type FormErrors = Record<string, string[]>
```

21. Как читать `keyof`

```
type UserKeys = keyof User
```

`keyof User` читается как объединение всех ключей типа `User`.

22. Как читать `T[K]`

```
type NameType = User['name']
```

23. Как читать сигнатуру `getProp`

```
function getProp<T, K extends keyof T>(obj: T, key: K): T[K]
```

24. Как читать mapped types в общем виде

```
type Flags<T> = {
  [K in keyof T]: boolean
}
```

25. Как читать условные типы

```
type IsString<T> = T extends string ? true : false
```

26. Самая важная психологическая мысль этапа

Страшная запись почти всегда собирается из знакомых маленьких кирпичей.

27. Как не тонуть в реальном коде

Алгоритм:

1. Найди внешнюю оболочку.
2. Найди базовую сущность.
3. Разбей на вложенные уровни.
4. Подставь конкретный пример вместо generic.
5. Распиши union отдельно.
6. Сформулируй тип обычным русским предложением.

28. Пример полного разбора сложного типа

```
type UserUpdateResult = Promise<
  | { ok: true; data: Readonly<Omit<User, 'passwordHash'>> }
  | { ok: false; error: string }
>
```

29. Как читать типы composable-like функций

```
function useAsyncData<T>(
  loader: () => Promise<T>
): {
  data: Ref<T | null>
  loading: Ref<boolean>
  error: Ref<string | null>
}
```

30. Как читать типы библиотек

Не надо понимать всю библиотечную типизацию. Достаточно локально расшифровать свой кусок использования.

31. Очень важный навык: переводить тип в обычный язык

```
function pick<T, K extends keyof T>(obj: T, keys: K[]): Pick<T, K>
```

Перевод: функция принимает объект и список его ключей, а возвращает новый объект только с этими ключами.

32. Когда нужно “развернуть тип руками”

Если тип слишком плотный, полезно временно переписать его в несколько строк.

33. Частые ошибки на этапе чтения сложных типов

Ошибка 1. Пытаться понять всё одним взглядом

Ошибка 2. Игнорировать базовый тип

Ошибка 3. Пугаться generic-букв

Ошибка 4. Путать union и объект

Ошибка 5. Не разворачивать utility types

Ошибка 6. Пытаться читать библиотечный тип целиком

34. Как тренировать этот навык

1. брать сложную сигнатуру и переписывать её человеческим языком;
2. разворачивать utility types вручную;
3. подставлять вместо generic конкретные типы.

35. Что особенно важно вынести из этапа 8

На этом этапе тебе не нужно стать мастером всех возможных conditional types. Нужно научиться:

- не бояться длинных типов;
- разбирать их по слоям;
- видеть базовую модель;
- читать utility types как операции преобразования;
- читать generics как шаблоны с подстановкой;
- читать union как набор вариантов;
- переводить сигнатуру на обычный язык.

36. Мини-конспект этапа

```
type UpdateUserDto = Partial<Omit<User, 'id' | 'createdAt'>>

type Result<T> =
  | { ok: true; data: T }
  | { ok: false; error: string }

type PaginatedResponse<T> = {
  items: T[]
  total: number
}

function getProp<T, K extends keyof T>(obj: T, key: K): T[K]

type UserData = Awaited<ReturnType<typeof loadUser>>
```

37. Самая короткая суть этапа

сложные типы — это почти всегда комбинации простых типов, и их нужно читать как набор маленьких операций, а не как одну страшную формулу.

38. Связь со следующим этапом

Этот этап особенно нужен перед Vue 3 + TypeScript, потому что в Vue ты начнёшь регулярно видеть:

- `Ref<T>`
- `ComputedRef<T>`
- `MaybeRef<T>`
- `PropType<T>`
- generic-composables
- типизацию `defineProps`
- типизацию `defineEmits`
- типизацию `watch`, `computed`, `reactive`
- пересечение типов из библиотек и собственного кода

Пользователь

Этап 9. Vue 3 + Composition API + TS

ChatGPT

Отлично. Это финальный этап, где вся предыдущая база начинает работать в реальном фронтенде.

Этап 9. Vue 3 + Composition API + TS — теория

На этом этапе задача уже не просто “знать TypeScript”, а **уметь применять его в Vue 3 без хаоса**.

И здесь есть очень важная мысль:

TypeScript в Vue нужен не для того, чтобы усложнить код, а для того, чтобы сделать понятными данные, пропсы, события, composables, API и состояние.

1. Главная цель этапа

Ты должен научиться типизировать:

- `props`
- `emits`
- `ref`
- `reactive`
- `computed`
- `watch`
- composables
- API-данные в компонентах
- формы
- списки
- template refs
- `v-model`
- состояния загрузки и ошибок

2. Главная ошибка новичка во Vue + TS

Самая частая ошибка — пытаться сразу понять сложные generic-типы Vue, `PropType`, типы библиотек и перегруженные сигнатуры.

3. Почему во Vue TypeScript особенно полезен

Во Vue очень много мест, где легко потерять форму данных.

4. Базовая форма работы: `script setup lang="ts"`

```
<script setup lang="ts">
const title = 'Hello'
</script>
```

5. `ref` — первая и самая важная конструкция

```
<script setup lang="ts">
import { ref } from 'vue'

const count = ref(0)
</script>
```

6. Что важно понимать про `ref`

В скрипте доступ идёт через `.value`, в template — обычно просто `count`.

7. Когда тип `ref` выводится автоматически

Часто Vue и TS сами нормально выводят тип.

8. Когда тип `ref` нужно указывать явно

```
const user = ref<User | null>(null)
```

9. Очень важная реальная модель для `ref`

```
type User = {
  id: number
  name: string
}

const currentUser = ref<User | null>(null)
```

10. `reactive` — реактивный объект

```
<script setup lang="ts">
import { reactive } from 'vue'

const form = reactive({
  name: '',
  email: '',
})
</script>
```

11. Когда удобнее `reactive`, а когда `ref`

Упрощённо: `ref` — для одиночных значений, `reactive` — часто для объектов и форм.

12. Типизация `reactive`

```
type UserForm = {
  name: string
  email: string
  age: number
}

const form = reactive<UserForm>({
  name: '',
  email: '',
  age: 0,
})
```

13. Важная ловушка `reactive`

Нужно понимать не только типы, но и поведение Vue.

14. `computed` — производное значение

```
<script setup lang="ts">
import { ref, computed } from 'vue'

const firstName = ref('Dima')
const lastName = ref('Sokolov')

const fullName = computed(() => `${firstName.value} ${lastName.value}`)
</script>
```

15. Когда тип `computed` указывать явно

Обычно TS хорошо выводит тип сам.

16. `watch` — наблюдение за изменениями

```
watch(count, (newValue, oldValue) => {
  console.log(newValue, oldValue)
})
```

17. Как мыслить типами в `watch`

Спрашивай себя:

- что именно я наблюдаю?
- какой тип у источника?
- возможен ли `null`?

18. `props` — один из важнейших участков типизации

```
<script setup lang="ts">
type Props = {
  title: string
  count?: number
}

const props = defineProps<Props>()
</script>
```

19. Почему типизация `props` так важна

Потому что пропсы — это входной контракт компонента.

20. `props` через type-based подход лучше для старта

Сначала думай не “как написать defineProps”, а “какой контракт у компонента”.

21. `emits` — типизация событий компонента

```
<script setup lang="ts">
type UserForm = {
  name: string
  email: string
}

const emit = defineEmits<{
  save: [payload: UserForm]
  cancel: []
}>()
</script>
```

22. Почему `emits` так важен

Он превращает события в нормальный контракт, как и `props`.

23. Связь `props` и `emits`

- `props` — что компонент принимает;
- `emits` — что отдаёт наружу.

24. `v-model` и типизация

`v-model` тоже упирается в props/emits.

25. Composable — это просто функция, но очень важная

```
function useCounter() {
  const count = ref(0)

  const increment = () => {
    count.value++
  }

  return {
    count,
    increment,
  }
}
```

26. Как правильно мыслить composable

Composable нужно типизировать так же, как обычную функцию.

27. Пример composable с API

```
type User = {
  id: number
  name: string
}

function useUsers() {
  const data = ref<User[]>([])
  const loading = ref(false)
  const error = ref<string | null>(null)

  return {
    data,
    loading,
    error,
  }
}
```

28. Более зрелый вариант: generic composable

```
function useAsyncData<T>() {
  const data = ref<T | null>(null)
  const loading = ref(false)
  const error = ref<string | null>(null)

  return {
    data,
    loading,
    error,
  }
}
```

29. Формы во Vue + TS

```
type UserForm = {
  name: string
  email: string
  age: number
}

const form = reactive<UserForm>({
  name: '',
  email: '',
  age: 0,
})
```

30. Ошибки формы

```
type FormErrors = Partial<Record<keyof UserForm, string>>
```

31. Списки во Vue + TS

```
type User = {
  id: number
  name: string
}

const users = ref<User[]>([])
```

32. Template refs

```
const inputRef = ref<HTMLInputElement | null>(null)
```

33. Почему template refs требуют `null`

Потому что элемент не существует мгновенно при создании скрипта.

34. `provide/inject`

`inject` часто может вернуть `undefined`, если значение не найдено.

35. API в компонентах

Компонент должен видеть уже осмысленные данные, а не сырую кашу.

36. Хорошая типовая модель async-состояния в Vue

```
type AsyncState<T> =
  | { status: 'loading' }
  | { status: 'success'; data: T }
  | { status: 'error'; message: string }

const state = ref<AsyncState<User[]>>({ status: 'loading' })
```

37. Что особенно важно во Vue: не терять честность типов

Если данных пока нет — значит `null`. Если идёт загрузка — это должно быть видно в типе.

38. Очень важная связь между Vue и предыдущими этапами

Vue + TS — это сборка всего, что ты прошёл раньше.

39. Как правильно изучать Vue + TS

Порядок такой:

1. `ref`, `reactive`, `computed`
2. `props`, `emits`
3. composables, API-состояния, формы
4. потом generic composables и сложные типы библиотек

40. Самые частые ошибки на этапе Vue + TS

Ошибка 1. Пихать всё в `any`

Ошибка 2. Не учитывать `null`

Ошибка 3. Использовать один тип на всё

Ошибка 4. Неправильно мыслить composables

Ошибка 5. Не типизировать `emits`

Ошибка 6. Бояться `Ref<T>`

Ошибка 7. Лезть сразу в сложные типы Vue-библиотек

41. Как мыслить правильно в компоненте

Когда пишешь Vue-компонент, полезно спрашивать себя:

1. Какие данные приходят извне?
2. Какие данные локальные?
3. Где нужен `ref`, а где `reactive`?
4. Может ли значение быть `null`?
5. Какие события компонент эмитит?
6. Что является формой?
7. Где у меня API-состояние?
8. Нужен ли composable?
9. Есть ли общий тип, который лучше вынести отдельно?

42. Мини-конспект этапа

```
type User = {
  id: number
  name: string
}

type Props = {
  user: User
  loading?: boolean
}

type UserForm = {
  name: string
  email: string
}

const props = defineProps<Props>()

const emit = defineEmits<{
  save: [payload: UserForm]
  cancel: []
}>()

const count = ref(0)
const currentUser = ref<User | null>(null)

const form = reactive<UserForm>({
  name: '',
  email: '',
})

const fullName = computed(() => {
  return currentUser.value ? currentUser.value.name : 'Guest'
})

const inputRef = ref<HTMLInputElement | null>(null)

type AsyncState<T> =
  | { status: 'loading' }
  | { status: 'success'; data: T }
  | { status: 'error'; message: string }

const usersState = ref<AsyncState<User[]>>({ status: 'loading' })
```

43. Что нужно вынести из этапа 9

После этого этапа ты должен понимать:

- как включить TypeScript в Vue через `lang="ts"`;
- как типизировать `ref` и когда нужен явный тип;
- как типизировать `reactive`-объекты и формы;
- как типизировать `computed`;
- как типизировать `props` через модель данных;
- как типизировать `emits`;
- как мыслить composables как обычные функции;
- как строить async-state во Vue;
- как типизировать template refs;
- как честно отражать `null`, `loading`, `error`, optional-поля;
- почему Vue + TS — это применение всей предыдущей базы, а не отдельная магия.

44. Самая короткая суть этапа

Этап 9 учит тебя вот чему:

Vue 3 + TypeScript — это не про сложные типы ради сложных типов, а про честные контракты данных внутри компонентов, composables и реактивного состояния.

45. Что у тебя теперь должно сложиться в голове

Вся дорога выглядит так:

- сначала ты учишься видеть формы данных;
- потом учишься описывать их типами;
- потом преобразовывать их;
- потом типизировать async и API;
- потом читать сложные сигнатуры;
- и только после этого спокойно переносишь всё в Vue 3 Composition API.

И это как раз правильный порядок.

Следующим полезным шагом будет уже не новая теория, а **собрать всё в одну цельную методичку по всем 9 этапам в компактном виде**.